Section 5.1: Cache Tier Architectures & Granular Interception Tiers

> ***Study Guide*** *| Part 1 of 4 | System Design Interview Prep Series*

Executive Overview

Caching is the art of trading memory for latency. A well-designed cache hierarchy can serve 99%+ of traffic without touching disk. This section covers the five canonical cache tiers, interception granularity patterns, and how to reason about cache design in system design interviews.

Core mental model: Every cache sits between a fast consumer and a slow producer. The goal is to push data as close to the consumer as possible, at the right granularity, with a manageable invalidation story.

The Five Cache Tiers

Cache tiers are ordered by distance from the user. Each hop closer to the user reduces latency but adds invalidation complexity.

```
User request flow:
Browser → CDN → Web Proxy → App Cache → DB Buffer → Disk

Typical hit rates:
  Browser:    30%   (user-controlled, cleared often)
  CDN:        60%   (geographic PoP nearest user)
  Web Proxy:  10%   (short TTL or low cache ratio)
  App Cache:  85%   (Redis/Memcached, explicit control)
  DB Buffer:  95%   (in-memory pages, automatic)

Compound miss rate: (0.70)(0.40)(0.90)(0.15)(0.05) ≈ 0.002
→ Only ~0.2% of requests reach disk
```

Tier 1 — Browser Cache

What it is: Storage local to the user's device. Controlled via HTTP headers, JavaScript APIs, or Service Workers.

Mechanism	Capacity	Use Case
`localStorage` / `sessionStorage`	5–10 MB	User preferences, tokens
`IndexedDB`	50 MB+	Offline-capable app data
HTTP Cache (`Cache-Control` , `ETag`)	Browser-controlled	Static assets, pages
Service Worker	Configurable	Full offline support

When to rely on it: Static assets (JS/CSS bundles, images), user preferences, offline-first applications.

Key limitation: User or browser can clear at any time. Capacity varies by browser and OS. Not suitable for sensitive or frequently-changing data.

Tier 2 — CDN Edge Cache

What it is: A geographically distributed network of caching servers (Points of Presence / PoPs) that serve content close to the user.

```
CDN PoP footprint (major providers):
  Cloudflare:  300+ PoPs
  Akamai:      4,000+ PoPs
  AWS CloudFront: 600+ PoPs

Per-PoP capacity: 100 TB+ storage, 10s of Gbps throughput
Typical latency: <10ms if content is cached at nearby PoP
```

Invalidation approaches: - **TTL expiration**: Simplest; staleness window = TTL - **Stale-While-Revalidate (SWR)**: Serve stale instantly, refresh in background - **Cache tag / surrogate keys**: Purge grouped content atomically (Cloudflare, Fastly)

When to use: Any static or semi-static content — HTML pages, images, video, API responses with low change frequency.

Key limitation: Propagation delays; emergency purges can take seconds to minutes globally.

Tier 3 — Web Server / Proxy Cache

What it is: An in-process or sidecar cache at the web/application server layer. Common implementations: NGINX upstream cache, Varnish Cache.

```
# NGINX proxy cache configuration
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=my_cache:10m max_size=1g;

location /api/ {
    proxy_cache my_cache;
    proxy_cache_key "$uri$is_args$args";
    proxy_cache_valid 200 60s;
    add_header X-Cache-Status $upstream_cache_status;
}
```

Cache key construction is critical: The key must capture every dimension that produces a different response — URL, query params, `Accept-Language`, authentication state.

When to use: Reducing load on origin for highly cacheable endpoints (product listings, public profiles, feed previews).

Key limitation: Single server = single point of failure unless replicated. Shared server state adds coordination complexity.

Tier 4 — Application Cache (Redis / Memcached)

What it is: An external, networked in-memory store. Applications read/write explicitly. The most flexible tier — the application controls keys, TTLs, and invalidation.

Feature	Redis	Memcached
Data types	Strings, Lists, Sets, Hashes, Sorted Sets, Streams	Strings only
Persistence	RDB snapshots + AOF log	None
Clustering	Redis Cluster (built-in sharding)	Client-side sharding
Pub/Sub	Yes	No
Throughput		~200K+ ops/sec multi-thread

Feature	Redis	Memcached
	~100K ops/sec single thread	
Best for	Complex data, persistence needed	Simple KV at extreme speed

When to use Redis: Session storage, leaderboards (Sorted Sets), pub/sub, rate limiting, complex aggregations.

When to use Memcached: Pure KV caching where persistence and types don't matter; multi-threaded throughput is the bottleneck.

Tier 5 — Database Buffer Pool

What it is: The DBMS's own in-memory page cache. Every read goes through the buffer pool; dirty pages are written back asynchronously.

```
Buffer pool sizing rule of thumb:
  OLTP workloads:  25-50% of available RAM
  Analytics/OLAP:  50-75% of available RAM

PostgreSQL: shared_buffers + OS page cache
MySQL InnoDB: innodb_buffer_pool_size

Hit ratio target: > 99% for production OLTP
Monitoring:
  PostgreSQL: SELECT * FROM pg_statio_user_tables;
  MySQL: SHOW STATUS LIKE 'Innodb_buffer_pool_read%';
```

Key limitation: Only accelerates the same database. Cross-service or cross-database caching requires Tier 4.

Cache Interception Granularity

Granularity answers: *What exactly do we cache — a raw SQL result, a computed object, or a rendered page?*

Query-Level Caching

Cache the result of a specific query text + parameters.

```
-- Query: SELECT * FROM users WHERE id = 123
-- Cache key: "query:users:id:123"

-- On UPDATE users SET name = 'Bob' WHERE id = 123
-- → Must invalidate all keys matching "query:users:id:123:*"
```

Fits when: The same query runs very frequently with the same parameters. Works well for read-heavy lookup patterns.

Breaks when: Query text varies (different column sets, joins, ordering). High write rate causes constant invalidation churn.

Object-Level Caching

Cache the deserialized domain object, independent of which query produced it.

```
# Cache the User domain object, not the SQL row
def get_user(user_id: int) -> User:
    cache_key = f"user:{user_id}"
    raw = redis.get(cache_key)
    if raw:
        return User.from_json(raw)

    # Any query path — ORM, raw SQL — can populate this
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)
    if user:
        redis.setex(cache_key, user.to_json(), 3600)
    return user
```

Fits when: Many code paths (REST endpoints, gRPC handlers, background jobs) all need the same entity. Granular per-entity TTL control.

Breaks when: Objects are large and partially updated frequently (forces full cache invalidation on any field change).

Granularity Selection Matrix

Consideration	Prefer Query Cache	Prefer Object Cache
Query variety	Low (few stable queries)	High (many code paths)
Write frequency	Low	Medium

Consideration	Prefer Query Cache	Prefer Object Cache
Object size	Any	Prefer smaller
Cross-service reuse	No	Yes
Invalidation precision	Per-query	Per-entity

Interview Question 1 — Profile Fan-Out Invalidation

Prompt: Design a cache invalidation system for a social media feed that maintains <1 second eventual consistency. How do you handle the fan-out when a user updates their profile?

Problem Analysis

```
Scale assumptions:
  Average followers: 500
  Celebrity (top 0.1%): 10M+ followers

Cache layout:
  user:{user_id}          → User profile object (TTL 1hr)
  feed:{follower_id}:{pg} → Paginated feed for each follower (TTL 5min)

Challenge: Invalidating 10M feed caches synchronously = 10M Redis DEL ops
  @ 100K ops/sec Redis → 100 seconds. WAY too slow.
```

Solution: Tiered Fan-Out by Follower Count

```
Follower count tiers:
  < 100 followers   → Synchronous invalidation (< 1ms overhead)
  100 – 10K         → Async batch via message queue (< 500ms)
  > 10K (celebrity) → Lazy invalidation + low TTL (< 60s staleness)
```

```
def update_profile(user_id: int, profile_data: dict):
    # 1. Write to DB (source of truth)
    db.update_user(user_id, profile_data)

    # 2. Invalidate own cache immediately
    redis.delete(f"user:{user_id}")

    # 3. Fan-out strategy by follower count
    follower_count = db.get_follower_count(user_id)

    if follower_count < 100:
```

```
        # Synchronous: small enough to block on
        for follower_id in db.get_followers(user_id):
            redis.delete(f"feed:{follower_id}:*")

    elif follower_count < 10_000:
        # Async batch: enqueue invalidation job
        queue.publish("feed_invalidation", {
            "user_id": user_id,
            "follower_ids": db.get_followers(user_id),
            "timestamp": time.time()
        })

    else:
        # Celebrity path: don't fan-out, use short TTL + delta cache
        # Followers get stale for up to 60s — acceptable for celebrities
        pass
```

Delta Cache Pattern (Key Insight)

Instead of caching the full rendered feed (with embedded user data), cache only the list of post IDs. Hydrate user data separately at read time.

```
feed:{follower_id}:{page}  →  [post_id_1, post_id_2, ..., post_id_20]
user:{user_id}             →  {name, avatar, bio, ...}

Profile update → Only invalidate user:{user_id}
Feed cache stays valid (it only contains IDs, not user data)
```

Why this works: The feed cache no longer embeds user objects, so a profile change only invalidates one key (the user), not N × pages feed caches.

Interview Question 2 — Multi-Tier Cache Design for an E-commerce Product Page

Prompt: Design the caching layer for a product detail page that must handle 50K RPS globally with <50ms p99 latency. Products update ~100 times per day.

Solution

```
Tier layout:
  CDN (Cloudflare):    TTL=60s, ~80% global hit rate
    ↓ miss
  NGINX proxy cache:   TTL=30s, per-datacenter
    ↓ miss
  Redis (object cache): TTL=300s, per-region cluster
```

```
    ↓ miss
  PostgreSQL + read replica: Source of truth

Update invalidation:
  On product write → Publish event to Redis Pub/Sub
  NGINX and Redis listeners → Delete their cached entries
  CDN → Cache-Tag purge via API (takes ~2s)

Cache key design:
  product:{product_id}:v{version}   (include version for instant invalidation)
  CDN surrogate key: "product-{product_id}"
```

Key trade-off: CDN purge takes 2–5 seconds globally. During this window, users may see stale price/inventory. Acceptable for display data; never acceptable for checkout — always hit the DB at purchase time.

Interview Question 3 — Read-Heavy User Profile Service

Prompt: You have a user profile service hitting 500K reads/second, 200 writes/second. P99 read latency must stay under 5ms. How do you cache?

Solution

```
Read:write ratio: 2500:1 → Extremely cache-friendly

Strategy: Cache-aside with Redis Cluster
  Read:  Check Redis → hit? return. miss? → fetch DB → populate cache
  Write: Update DB → delete cache key (invalidate, not update)

Why delete instead of update on write?
  Write-through adds latency to every write.
  With 200 writes/sec, 200 extra Redis writes acceptable.
  But avoiding read-modify-write race conditions is simpler with delete.

TTL: 1 hour (200 writes/24hr = ~8 updates/user/day; 1hr staleness acceptable)
Cache miss budget: With 99% hit rate, 500K × 0.01 = 5K DB reads/sec
  → Well within DB capacity

Redis cluster sizing:
  Avg profile size: 2KB
  Active users: 5M (80/20 rule: cache 1M active users)
  Memory: 1M × 2KB = 2GB + 30% overhead ≈ 3GB
  → Single Redis node sufficient; cluster for HA
```

Summary: When to Use Each Cache Tier

Scenario	Recommended Tier(s)
Static JS/CSS assets	Browser + CDN
Public product pages, blog posts	CDN + Web Proxy
Authenticated user data	App Cache (Redis)
Aggregated/computed results	App Cache
Database hot rows (same DB)	DB Buffer Pool
Cross-service shared state	App Cache
Offline-capable mobile/web app	Browser (Service Worker)